

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Technology (CSE)

QUIZ 1**DURATION: 20 MINUTES****SUMMER SEMESTER, 2024-2025****FULL MARKS: 15**

SWE 4201: Object Oriented Concepts I

Programmable calculators are not allowed.

Answer **all 3 (three)** questions. Figures in the right margin indicate full marks of questions whereas corresponding CO and PO are written within parentheses.

1. Why does Java require explicit type casting when converting from `double` to `int`?

2.5
(CO1)
(PO1)

Solution:

Java requires explicit type casting when converting from `double` to `int` because it is a narrowing conversion. A `double` can store decimal values and a much larger range of numbers, while an `int` can only store whole numbers.

During conversion, the fractional (decimal) part is discarded, which may lead to loss of data. To prevent accidental data loss, Java forces the programmer to perform the conversion explicitly.

For example:

```
1 double x = 15.99;  
2 int y = (int) x;
```

After conversion, `y` becomes 15 because the decimal part `.99` is removed.

2. What advantages does constructor overloading provide in object-oriented programming?

2.5
(CO1)
(PO1)

Solution:

Constructor overloading allows a class to have multiple constructors with different parameter lists. This enables objects of the same class to be initialized in different ways depending on the available information.

It improves flexibility, readability, and convenience because users can create objects using different sets of values without writing separate methods.

For example, one constructor may initialize only a name, while another may initialize both name and age.

3. Consider the following program and answer the questions briefly:

10
(CO2)
(PO2)

```
1 class Product {  
2  
3     String name;  
4     double price;  
5     static int totalProducts = 0;  
6  
7     Product(String name, double price) {  
8         this.name = name;  
9         this.price = (int) price;
```

```

10         totalProducts++;
11     }
12
13     void applyDiscount(int percent) {
14         int discount = (int) (price * percent / 100);
15         price = price - discount;
16     }
17
18     void display() {
19         System.out.println(name + " " + price + " " + totalProducts);
20     }
21 }
22
23 public class Main {
24
25     public static void main(String[] args) {
26
27         Product p1 = new Product("Mouse", 999.99);
28
29         Product p2 = p1;
30
31         p2.applyDiscount(15);
32
33         Product p3 = new Product("Keyboard", 500.50);
34
35         System.out.println(p1.price);
36         System.out.println(p2.price);
37         System.out.println(Product.totalProducts);
38     }
39 }

```

- What is the output of the program?
- Does the program correctly calculate discounted price? Explain.
- How many objects are actually created? Explain carefully.
- What is the value of `totalProducts` and why?
- Modify the code so that applying discount on `p2` does NOT affect `p1`. Write only the corrected statement(s).

Solution:

a) Output:

```

1 850.0
2 850.0
3 2

```

Explanation:

- `999.99` becomes `999` after explicit casting.
- But since `price` is a double variable, the `int` is implicitly converted back to double -> `999.0`.
- `Discount = (999.0 × 15)/100 = 149.85`. After casting to `int`, `discount = 149`

- Final price = $999.0 - 149 = 850.0$
- p2 points to the same object. Applying discount to p2 affects p1 and vice versa. Hence both prices are 850.0.
- p2 is not a new object, so totalProducts = 2.

b) The discounted price is not fully accurate because:

- price loses its fractional value when converted into integer.
- Decimal precision is lost during explicit casting in discount calculation.

c) Only TWO objects are created:

- One object for p1
- One object for p3

p2 = p1 only creates another reference to the same object.

d) totalProducts = 2 because:

- totalProducts is static and shared among all objects.
- Only two constructor calls create objects.

e) Corrected statement:

```
1 Product p2 = new Product(p1.name, p1.price);
```

OR

```
1 Product p2 = new Product("Mouse", 999.99);
```

Now p2 refers to a separate object. Applying discount on p2 will no longer affect p1.